Programa:	Visual Day Planner 7.3	
Protección:	Time-Trial + Nagscreen	
Objetivos:	Evitar todas las limitaciones	
URL descarga:	www.inklineglobal.com	
Dificultad:	Muy fácil	
Herramientas:	OllyDbg (opcional) + SmartCheck.	
Cracker:	SyXe'05 [hAcK-c0d3d] [CracksLatinos]	Fecha: 14-09-2005

Hola chavales..!!

Espero que tengamos hoy un rato animado entre nosotros, veremos como un programa nos regala cientos de claves válidas para registrar un producto por el que nos querían "robar" 29.95$ pobre del que los pague, jeje..

Bueno como siempre echaremos un vistazo con el PEiD a ver que nos puede decir:

Microsoft Visual Basic 5.0 / 6.0

Vale Visual Basic, un lenguaje que puede ser tan fácil como tan difícil, es extremadamente muy algo, cargamos el programa en Olly y damos RUN a ver que tal, aparece una nag diciendo que no estamos registrados y que nos quedan 30 días (lo acabamos de instalar), acto seguido aparece el programa, en el About no pone nada, no vemos opción de registro ¿? Vale lo cerramos y de repente otra nag, joder que pesaditos, la cerramos por su puesto.

Tenemos a simple vista un par de nags que superar, cogemos el Olly parcheado para encontrar OEP'S ya que tiene la propiedad de sólo detenerse en los BPM por ejecución 8ni por lectura ni escritura) esto nos va bien para cuando estemos en la librería de visual ponerlo en la primera sección del .exe y dar Run así sólo estaremos en el ejecutable sin tracear la .dll, cosa de agradecer teniendo en cuenta todo lo que para en ella.

Siguiendo este método no es difícil encontrar la primera nag, pues comienza a asomar la cabeza aquí (aunque queda transparente):

```
0054AE1D   .  FF90 B0020000 CALL DWORD PTR DS:[EAX+2B0]              ;  MSVBVM60.66104348
```

aún no se ve pero se aprecia la silueta, o sea que está en pantalla, después decidirá que texto insertar en ella si el del Trial o el de Expired, nos hace una jugarreta rara en este CALL que evita pasar por un buen salto que evita ambas zonas:

```
0054B593   .  50            PUSH EAX
0054B594   .  FF15 24124000 CALL DWORD PTR DS:[<&MSVBVM60.__vbaR8Str>;  MSVBVM60.__vbaR8Str
```

En este CALL ya digo que hace algo raro, bueno nos mete en una excepción y de repente caemos en una zona que sobrepasa este salto de largo (el SEH nos pone aquí directamente):

```
0054B61F   .  83C4 14       ADD ESP,14
0054B622   .  0FBF8D 7CFEFF>MOVSX ECX,WORD PTR SS:[EBP-184]
0054B629   .  85C9          TEST ECX,ECX
0054B62B   .  0F84 82070000 JE 2day.0054BDB3               ;Bonito SALTO..!!
0054B631   .  C745 FC 20000>MOV DWORD PTR SS:[EBP-4],20    ;Caemos justo aquí..!!
0054B638   .  E8 E3C6FFFF   CALL 2day.00547D20
0054B63D   .  66:8945 C8    MOV WORD PTR SS:[EBP-38],AX
0054B641   .  C745 FC 21000>MOV DWORD PTR SS:[EBP-4],21
0054B648   .  66:837D C8 00 CMP WORD PTR SS:[EBP-38],0
0054B64D   .  0F8F 91010000 JG 2day.0054B7E4               ;Controla el Trial
```

Caemos cerca del segundo salto marcado JG ya que con un BP allí para pero no en el JE anterior, esto lo hará porque sabrá que no estamos registrados seguramente, en realidad lo que hace, ya que traceando un poco se sabe, es levantar una excepción, si miramos el SEH Chain veremos donde tiene el manejador instalado, podemos poner un BP allí y tracearlo, pues básicamente lo que ocurre es que se le pasa al CALL responsable de esto una cadena alfanumérica tipo EXTENCOD-9818-05098, seguramente este es el código por defecto, al entrar en el mencionado CALL comprueba esta cadena y al ver esta crea la excepción, probamos a cambiarla por 121212 y a excepción no ocurre, esto es la prueba que es cierto lo que pensamos, vale, una vez dentro del manejador realiza un par de operaciones y nos envía de directamente hacia :54B62B sin pasar por el código anterior por eso los BP's no surten efecto. Todo esto lo comento como observación porque no tiene ningún efecto relevante en lo que estmamos mirando, simplemente me llamó la atención lo que hacía y por eso os lo comento.

Sabemos que la nagscreen hace acto de presencia en el CALL DWORD PTR DS:[EAX+2B0] que vimos antes, pero lo hace aún en modo transparente (no está visible), y sabemos que está por el pantallazo que da, si miramos el valor de ESP vemos que aumenta +24 uds. tras pasar por este CALL, entonces podemos cambiar la instrucción:

```
0054AE1D      FF90 B0020000 CALL DWORD PTR DS:[EAX+2B0]     POR:
0054AE1D      83C4 24       ADD ESP,24
0054AE20      90            NOP
0054AE21      90            NOP
0054AE22      90            NOP
```

Y evitamos la nagscreen, jeje.. ahora queda otra nag y evitar que expire el programa, la otra nag la eliminamos poniendo un BP DestroyWindow cuando esté en pantalla, la cerramos y caemos en la librería de Visual, ponemos un BPM en la primera sección del programa y damos RUN, al momento caemos en esta zona ya del propio ejecutable:

```
0052F6A3   .  FF51 10       CALL DWORD PTR DS:[ECX+10]            ; MSVBVM60.660C886A
0052F6A6   .  DBE2          FCLEX                                 ; Salimos aquí
0052F6A8   .  8985 2CFFFFFF MOV DWORD PTR SS:[EBP-D4],EAX
```

O sea que la llamada se realizó desde :52F6A6, ponemos un BP en esta dirección y reiniciamos, damos RUN y cuando salga el programa lo cerramos, sale la nag, la cerramos y salta nuestro BP, o sea que es un poco tarde no importa, el retorno nos lleva a la librería de visual nuevamente pero ponemos el BPM y caemos en:

```
0052E45C   .  FF91 B0020000 CALL DWORD PTR DS:[ECX+2B0]           ; MSVBVM60.66104348
0052E462   .  DBE2          FCLEX                                 ; Salimos aquí
0052E464   .  8985 18FFFFFF MOV DWORD PTR SS:[EBP-E8],EAX
```

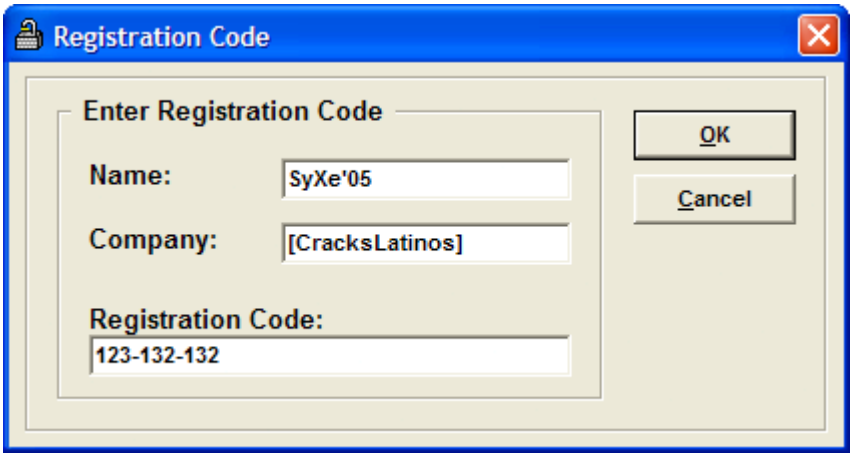
Repetimos la jugada, ponemos el BP en :52E45C y reiniciamos, al cerrar el programa ahora sí salta el BP y la nag no ha salido, jeje.. nos hemos anticipado, por si acaso tomamos el valor de la pila (registro ESP de 32 bits) tenemos:

```
Antes del CALL => ESP==12EE38
Después del CALL => ESP==1212EE5C
Diferencia => ESP+24
```

Como ya viene siendo habitual hay un desajuste de +24 unidades en este registro (puntero de pila = Stack Pointer), entonces significa que podemoa cambiar:

```
0052E45C   .  FF91 B0020000 CALL DWORD PTR DS:[ECX+2B0]     POR:
0052E45C      83C4 24       ADD ESP,24
0052E45F      90            NOP
0052E460      90            NOP
0052E461      90            NOP
```

Y ya entramos y salimos sin nag. Ahora queda adelantar el reloj del sistema y lanzar la aplicación a ver si hay algún problema… lo adelantamos 1 año que es más cómodo y ejecutamos el exe sin modificar, para nuestra sorpresa nos deja entrar en el programa y ahora nos da la opción de introducir un serial, vale vamos a verlo



Aceptamos y nos meten el broncón padre, jeje..

Vale ahora podemos utilizar el SmartCheck y ver si encontramos el zerial válido,

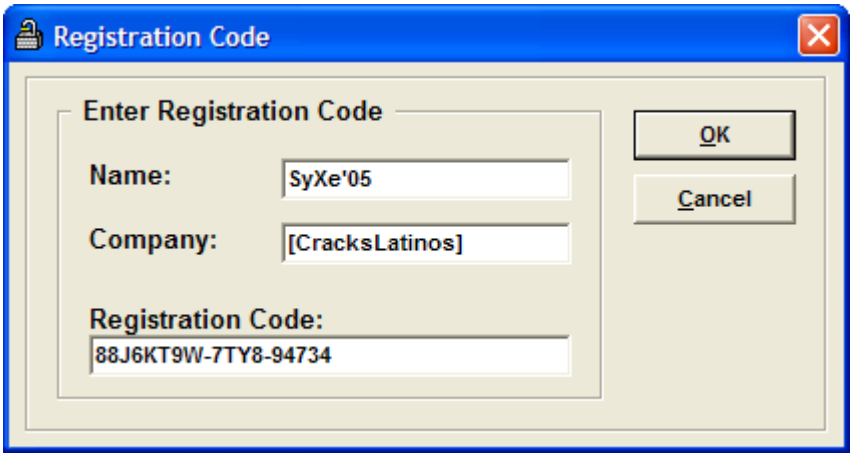
```
45568   RegcodeForm_KeyPress
45569   tmMidnightRefresh_Timer
45573   RegcodeForm_KeyPress
45574   tmAlarm_Timer
45577   btnOK_Click
45578     OnError(long:1)
45579     tbCode.Text
45580     tbCode.Text
45581     LCase$(String:Null)
45582     LCase$(String:"123-123-...")
45583     tbCode.Text
45584     LCase$(String:"88J6KT9W...")
45585     LCase$(String:"123-123-...")
45586     LCase$(String:"88J6KT9W...")
45587     LCase$(String:"123-123-...")
45588     LCase$(String:"88J6KT9W...")
45589     LCase$(String:"123-123-...")
45590     LCase$(String:"88J6KT9W...")
45591     LCase$(String:"123-123-...")
45592     LCase$(String:"88J6KT9W...")
45593     LCase$(String:"123-123-...")
45594     LCase$(String:"88J6KT9W...")
45595     LCase$(String:"123-123-...")
45596     LCase$(String:"88J6KT9W...")
45597     LCase$(String:"123-123-...")
45598     LCase$(String:"88J6KT9W...")
45599     LCase$(String:"123-123-...")
45600     LCase$(String:"88J6KT9W...")
45601     LCase$(String:"123-123-...")
45602     LCase$(String:"88J6KT9W...")
45603     LCase$(String:"123-123-...")
45604     LCase$(String:"88J6KT9W...")
45605     LCase$(String:"123-123-...")
45606     LCase$(String:"88J6KT9W...")
45607     LCase$(String:"123-123-...")
45608     LCase$(String:"88J6KT9W...")
```

¿Qué es esto? Son todos seriales válidos ¿? Todavía hay más pero no los voy a poner todos, aunque comienzan todos igual no son los mismos pues varía el final, mirad un ejemplo:

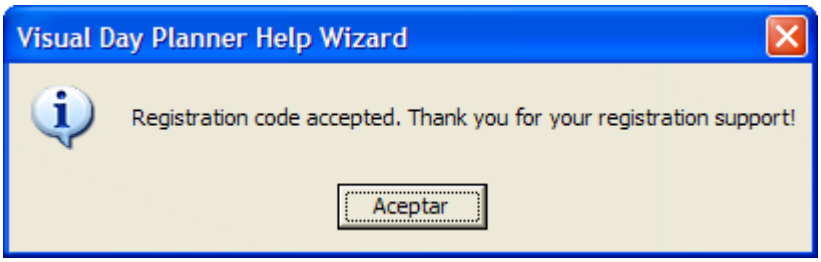
```
String string = 00186C84
    = "88J6KT9W-7TY8-94734"
```

```
String string = 00186C84
    = "88J6KT9W-3TK4-98G3G"
```

Como vemos son los dos distintos, lo mismo ocurre con cada uno de ellos, perplejos todavía probamos yo que sé el primero de ellos "88J6KT9W-7TY8-94734" por decir algo:



Probamos con estos datos, el nombre y la compañía es bueno que no varíen porque lo más normal es que el serial se genere de acuerdo a estos datos, si los variamos seguramente el serial será diferente por eso lo dejamos igual todo menos el serial, aceptamos y…



Pues vaya una lástima, ahora que nos estábamos divirtiendo resulta que nos regalan un número de serie, bueno a falta de uno quinientos, jajaja.. espectacular. La eliminación de las nags las hemos hecho gratis pero bueno la práctica hace al maestro dicen no?, así que no está de más, el trabajo bien hecho siempre está bien hecho.

Una bonita práctica en un programa compilado en Visual Basic de los que cada día se ven menos (o al menos en mi caso) pero que a veces nos dan tanta guerra.

Como comentario decir que aún seguimos viendo la nag inicial aunque ahora nos dice que estamos registrados, ya sabéis al que no le guste como hacerla desaparecer que no es difícil.

Y hasta aquí el trabajo de hoy, espero que haya sido ilustrativo y quede claro lo que he intentado explicar, para cualquier cosa me encontraréis en:

syxe00@yahoo.es
syxe00@hotmail.com